Рекуррентные нейронные сети

Математические основы и применения

Что такое RNN?

Определение

RNN — класс нейронных сетей для обработки **последовательных данных**

Ключевые особенности

- **Внутренняя память** — h_t
- **Разделяемые веса**
- Обработка произвольной длины

Примеры данных

- Тексты (слова)
- Речь (аудио)
- Временные ряды
- Видео (кадры)
- ДНК

Проблема обработки последовательностей

Стандартные нейросети

Ограничения:

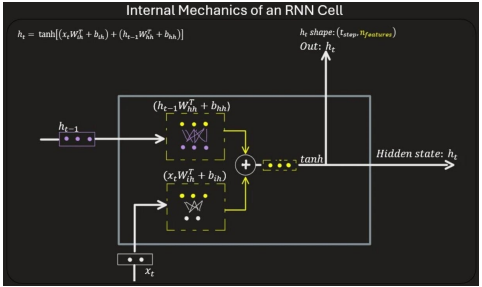
- Фиксированный размер входа
- Нет памяти
- Разные веса
- Не учитывают контекст

RNN

Преимущества:

- Переменная длина
- Внутреннее состояние
- Разделяемые веса W, b
- Учитывают контекст

Архитектура ячейки RNN



Компоненты

Входы:

- x_t — текущий вход
- h_{t-1} — прошл. состояние

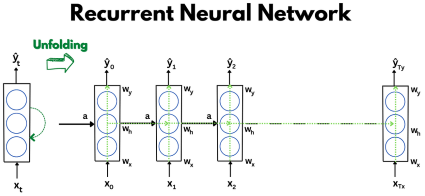
Параметры:

- W_{xh}, W_{hh} — веса
- b_h — смещение

Выход:

- h_t — новое состояние

Математика базовой RNN



Формулы Vanilla RNN

Скрытое состояние:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

Выход:

$$y_t = W_{hy}h_t + b_y$$

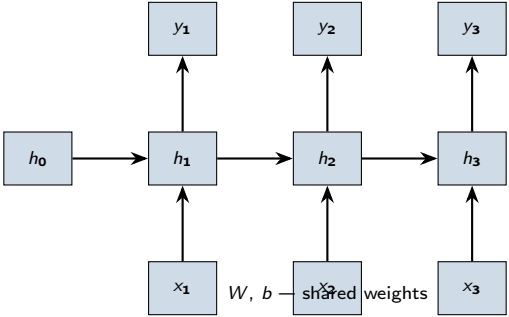
Ключевая особенность

Параметры W, b **разделяются** на всех шагах.

Развертывание сети во времени

Концепция

RNN разворачивается во времени — каждый шаг обрабатывается отдельной копией с **одинаковыми весами**.



Обучение: Backpropagation Through Time

Алгоритм BPTT

1. **Forward Pass:** Обработка шаг за шагом
2. **Loss:** Вычисление ошибки $L = \sum_{t=1}^{T} L_t$
3. **Backward:** Обратное распространение от T к 1
4. **Update:** Обновление весов по сумме градиентов

Правило цепочки

$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial h_{t-1}} \cdot \frac{\partial h_{t-1}}{\partial W}$$

Truncated BPTT

Для длинных последовательностей: window size $k \approx 20 - 50$.

Проблема исчезающего градиента

Математическая причина

$$\frac{\partial h_t}{\partial h_j} = \prod_{k=j+1}^{t} W_{hh}^T \cdot \mathrm{diag}(f'(z_k))$$

Условие

Если $\max |\lambda(W_{hh})| < 1$:

$$\left\| \frac{\partial h_t}{\partial h_j} \right\| \to 0$$

Последствия

- Градиенты исчезают
- Веса не обновляются
- Забывает контекст

Проблема взрывающегося градиента

Взрывающийся градиент

Причина: $\max|\lambda(W_{hh})| > 1$

$$\|\nabla h_t\| \propto \lambda^t$$

Последствия:

- NaN
- Нестабильно
- Резкие изменения

Gradient Clipping

Алгоритм:

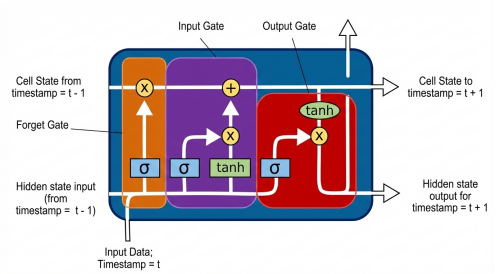
1. Градиент g
2. Если $\|g\| > threshold$:

$$g = \text{threshold} \cdot \frac{g}{\|g\|}$$

3. $W = W - \eta g$

threshold $\in [1, 5]$

Долгая краткосрочная память (LSTM)



Компоненты

Cell State (C_t):

- Конвейер информации
- Прямой поток градиента

Три вентиля:

1. Forget (f_t)
2. Input (i_t)
3. Output (o_t)

Преимущество

Решает исчезновение через **аддитивное обновление**.

Математика LSTM: Вентили

Forget Gate

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

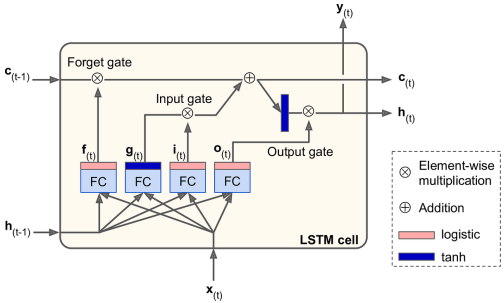
Input Gate

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad \tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

Output Gate

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Математика LSTM: Состояние и Выход



Обновление ячейки

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Скрытое состояние

$$h_t = o_t \odot \tanh(C_t)$$

Преимущество

Градиент протекает через C_t.

Gated Recurrent Unit (GRU)

Преимущества

- Меньше параметров
- Быстрее обучение
- Меньше переобучение

Компоненты

Два вентиля:

1. Update (z_t)
2. Reset (r_t)

Единое состояние: h_t

Сравнение

	LSTM	GRU
Вентили	3	2
Парам.	Больше	Меньше
Скорость	Медл.	Быстр.

Математика GRU

Reset Gate

$$r_t = \sigma(W_r[h_{t-1}, x_t])$$

Update Gate

$$z_t = \sigma(W_z[h_{t-1}, x_t])$$

Обновление состояния

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t, \quad \tilde{h}_t = \tanh(W[r_t \odot h_{t-1}, x_t])$$

Сравнение архитектур

Сравнительная таблица

Характ.	Vanilla	LSTM	GRU
Параметры	Мин.	Макс.	Средн.
Скорость	Быстро	Медл.	Быстро
Память	Плохо	Отлично	Хорошо
Градиент	Проблема	Решена	Решена
Сложность	Низк.	Выс.	Средн.

Vanilla

Академический интерес, редко на практике

LSTM

Стандарт для сложных задач

GRU

Баланс качества и скорости

Применение в NLP

Тональность

Many-to-One

- "Love"$\rightarrow$ POS
- "Bad"$\rightarrow$ NEG

Перевод

Seq2Seq

1. Encoder
2. Context
3. Decoder

Генерация

Language Models

- Следующее слово
- Чат-боты

Временные ряды и Речь

Прогнозирование

- Цены акций
- Погода
- Нагрузка

Many-to-Many

Распознавание речи

Аудио $\rightarrow$ Текст

- Ассистенты
- Субтитры

Музыка

Many-to-Many

- WaveNet
- Music Trans.

Продвинутые архитектуры

Bidirectional

Оба направления:

- Forward
- Backward
- Concat

Deep RNN

Несколько слоев:

- Иерархия
- Сложн. признаки

Attention

Важные части:

- Веса α_i
- Контекст

$\rightarrow$ Transformers

Ограничения и современные тенденции

Ограничения

1. **Последовательность**
   - Без распараллеливания
   - Медленное обучение
2. **Контекст**
   - Теряют память

Тенденции

Transformers:

- Распараллеливание
- Self-Attention

Модели: BERT, GPT

Где RNN полезны:

- Stream
- Edge
- Real-time

Заключение

Ключевые выводы

1. **RNN** ввели память в нейронные сети
2. **LSTM, GRU** решили проблему градиентов
3. **Attention** → Transformers
4. RNN заложили основы последовательных моделей

Будущее

Принципы RNN остаются важными:

- Последовательные данные
- Механизмы внимания
- Баланс память/вычисления

Спасибо за внимание!